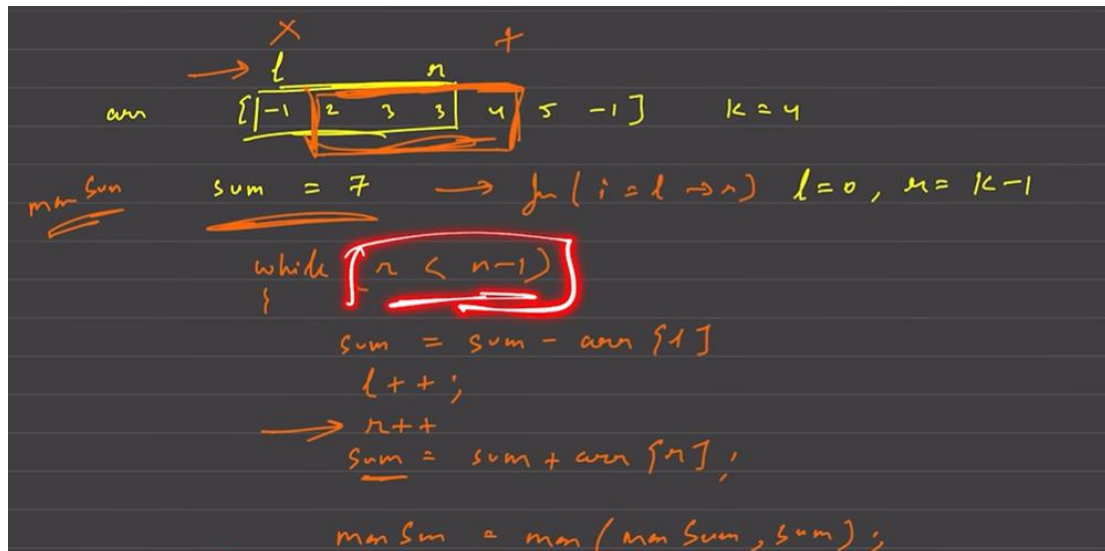


Patterns

1. Constant Window:

- Maximum sum we can get by picking 'k' elements consecutively from an array. Works for array contains +ve, -ve & zero.



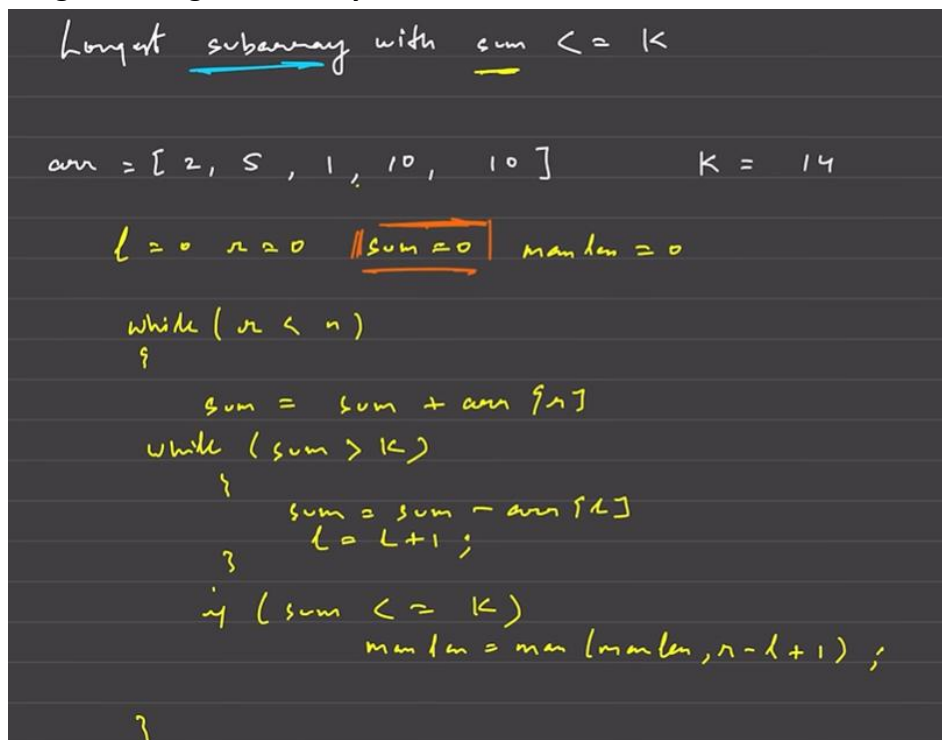
Handwritten code for the Constant Window problem:

```
arr = [-1, 2, 3, 3, 4, 5, -1]    k = 4
sum = 7
maxSum = 7
for (i = 0; i < n; i++) {
    sum = sum - arr[i];
    i++;
    sum = sum + arr[i];
    maxSum = max(maxSum, sum);
}
```

The code illustrates a sliding window of size k=4. The initial window is [2, 3, 3, 4] with a sum of 7. The code then iterates through the array, updating the sum by removing the leftmost element and adding the next element in the window.

2. Longest subarray or sub-string with condition

- Length of Longest subarray with $\text{sum} \leq k$



Handwritten code for the Longest subarray with sum $\leq k$ problem:

```
Longest subarray with sum <= k
arr = [2, 5, 1, 10, 10]    k = 14
l = 0, r = 0, sum = 0, maxlen = 0
while (r < n) {
    sum = sum + arr[r];
    while (sum > k) {
        sum = sum - arr[l];
        l = l + 1;
    }
    if (sum <= k) {
        maxlen = max(maxlen, r - l + 1);
    }
    r++;
}
```

The code uses a sliding window approach. It starts with l=0 and r=0. It expands the window by moving r forward until the sum exceeds k. Then, it contracts the window by moving l forward until the sum is less than or equal to k. The maximum length of the valid window is stored in maxlen.

Here time complexity= $O(2n)$, If the question will ask for the subarray we cannot optimize it more... But if not we can optimize it to $O(n)$.

```

✓ while (r < n)
{
    sum = sum + arr[r]
    if (sum > k)
    {
        sum = sum - arr[l]
        l = l + 1
    }
    if (sum <= k)
        maxlen = max(maxlen, r - l + 1);
    r = r + 1;
}

print (maxlen)

```

condition

Store l, n.

TC $\rightarrow O(N)$
 SC $\rightarrow O(1)$

3. No of subarray with some condition

- No of subarray with sum=k

4. finding minimum window with some condition

Medium Problems

Given a binary array nums and an integer k, return the maximum number of consecutive 1's in the array if you can flip at most k 0's.

Or

Length of Longest subarray with at most k zeroes.

💡 Core Intuition

Instead of actually flipping zeroes:

- Treat every zero inside the window as a "potential flip".
- Keep expanding the window as long as the number of zeroes $\leq k$.
- If zeroes exceed k , the window becomes invalid and must be adjusted.

Key Observation

A window is valid if:

- Number of zeroes inside it $\leq k$

Since we only need the maximum length, whenever the window is valid we can consider its size as a candidate answer.

Solution 1: Regular Sliding Window (optimal1)

Idea

Maintain:

- Left pointer
- Right pointer
- Count of zeroes in current window

Expand the right pointer.

Whenever zero count exceeds k :

- Keep moving the left pointer until the window becomes valid again.

After obtaining a valid window:

- Update maximum length.

Complexity Analysis

Time Complexity

$O(n)$

Reason:

- Right pointer moves n times.
- Left pointer also moves at most n times overall.

Space Complexity

$O(1)$

Only a few variables are used.

Solution 2: Optimized Sliding Window Observation (optimal2)

Idea

Observation:

When zero count exceeds k:

- We don't need to repeatedly shrink until valid.
- Shrinking by one position is enough.

Reason:

- Window size is only increasing gradually.
- We only care about the maximum possible size.
- Any oversized invalid window will eventually become valid as the left pointer keeps moving in future iterations.

Complexity Analysis

Time Complexity

$O(n)$

Reason:

- Both pointers move forward only.

Space Complexity

$O(1)$

Only constant extra memory.

Given a string s consisting only of characters a , b and c .

Return the number of substrings containing at least one occurrence of all these characters a , b and c .

Input: $s = \text{"abcabc"}$

Output: 10

Explanation: The substrings containing at least one occurrence of the characters a , b and c are "abc", "abca", "abcab", "abcabc", "bca", "bcab", "bcabc", "cab", "cabc" and "abc" (again).

Core Intuition

Instead of generating all substrings:

Think from the perspective of the **ending index**.

For every position:

- Treat it as the ending point of a substring.
- Determine how many substrings ending here contain a , b , and c .

Key Observation

For a substring ending at the current position to contain all three characters:

- It must start before or at the earliest among the latest occurrences of a , b , and c .

Why?

Because the substring must include the most recent occurrence of every required character.

The earliest of those three positions becomes the limiting factor.

Every starting position before or at that index forms a valid substring.

Solution: Last Occurrence Tracking

Idea

Maintain:

- Latest index of a

- Latest index of b
- Latest index of c

While traversing:

- Update the latest position of the current character.
- Once all three characters have appeared at least once:
 - Find the earliest among their latest positions.
 - All starting positions up to that index create valid substrings ending at the current position.

Add those valid starting positions to the answer.

Why It Works

Suppose:

- Latest a = 5
- Latest b = 8
- Latest c = 10

The earliest among them is 5.

Any substring ending at 10 must start at:

- 0, 1, 2, 3, 4, or 5

to include all three latest occurrences.

Starting after 5 would exclude the latest a, making the substring invalid.

Thus every position up to the minimum latest occurrence contributes a valid substring.

Complexity Analysis

Time Complexity

$O(n)$

Reason:

- Single traversal of the string.
- Constant work at each position.

Space Complexity

$O(1)$

Reason:

- Only three indices are stored.

You are given a string s and an integer k . You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times.

Return the length of the longest substring containing the same letter you can get after performing the above operations.

Core Intuition

For any window:

A A B A B

Suppose frequency of characters is:

- $A = 3$
- $B = 2$

To make the entire window consist of the same character:

- Keep the character with highest frequency (A).
- Replace all other characters.

Required replacements:

Window Length - Maximum Frequency Character

Example:

A A B A B

Length = 5

Max Frequency = 3

Replacements Needed = $5 - 3 = 2$

If replacements needed $\leq k$, then the window is valid.

Why does this work?

To maximize window size, we should keep the character that already appears the most and change all remaining characters.

Any other choice would require more replacements.

Approach 1: Brute Force

Idea

For every starting index:

- Extend the substring.
- Track frequencies.
- Track maximum frequency.
- Calculate replacements needed.
- Update answer if valid.

Complexity

- Time: $O(n^2)$
- Space: $O(1)$

Approach 2: Sliding Window + Recompute maxFreq

Idea

Maintain a window.

While window becomes invalid:

- Shrink from left.
- Recompute maxFreq by scanning frequency array.

Why?

Shrinking may remove the character contributing to maxFreq, so we recalculate.

Complexity

- Time: $O(26 \times n) \approx O(n)$
- Space: $O(1)$

Approach 3 (Optimal): Sliding Window + Never Decrease maxFreq

Key Trick

While expanding:

- Update maxFreq normally.

While shrinking:

- Do NOT recalculate maxFreq.
- Do NOT decrease maxFreq.

Why does it work?

Even if maxFreq becomes stale:

- It may delay shrinking.
- It never causes us to miss the optimal answer.
- Both pointers still move only forward.

This converts the solution into pure linear time.

Complexity

- Time: $O(n)$
- Space: $O(1)$

Given a binary array `nums` and an integer `goal`, return *the number of non-empty **subarrays** with a sum goal*.

A **subarray** is a contiguous part of the array.

Or

Given an array of integers `nums` and an integer `k`. A continuous subarray is called **nice** if there are `k` odd numbers on it.

Return *the number of **nice** sub-arrays*.

Brute-Force

Generate every possible subarray.

For each starting index:

- Keep extending the subarray.
- Continuously calculate its sum.
- Whenever the sum becomes equal to goal, increase the answer.

Since every subarray is checked, no valid answer can be missed.

Complexity

Time: $O(n^2)$

Space: $O(1)$

Approach 2: Prefix Sum + HashMap

Core Intuition

Instead of recalculating every subarray sum:

- Maintain the running sum from the beginning.
- For the current position, determine how many previous prefix sums can form a subarray with sum = goal.

Think:

If current prefix sum is S ,
then we need an earlier prefix sum equal to:

Current Sum – Goal

Every occurrence of that required prefix sum forms one valid subarray ending at the current index.

Store frequencies of prefix sums in a HashMap.

Complexity Analysis

Time Complexity

- Single traversal.
- HashMap operations are constant time on average.
- $O(N)$

Space Complexity

- Prefix sums are stored in HashMap.
- $O(N)$

Solution 3: Sliding Window (Optimal for Binary Arrays)

Core Intuition

Direct sliding window cannot count subarrays with **exactly** sum = goal.

Instead, use a transformation:

Count:

- Subarrays with sum $\leq$ goal
- Subarrays with sum $\leq$ goal-1

Their difference gives:

Subarrays with sum exactly = goal

Why This Transformation Works

All subarrays counted in:

sum $\leq$ goal

contain:

- sums smaller than goal
- sums equal to goal

All subarrays counted in:

sum $\leq$ goal-1

contain only:

- sums smaller than goal

Subtracting removes all smaller sums, leaving only:

sum = goal

Complexity Analysis

Time Complexity

- Each pointer moves at most N times.
- Helper = **$O(N)$**
- Called twice.

Overall: **$O(N)$**

Space Complexity

- Only variables and pointers are used.
- **$O(1)$**

HARD PROBLEMS

Given an integer array `nums` and an integer `k`, return *the number of **good subarrays** of `nums`.*

A **good array** is an array where the number of different integers in that array is exactly `k`.

- For example, `[1,2,3,1,2]` has 3 different integers: 1, 2, and 3.

A **subarray** is a **contiguous** part of an array.

Core Intuition

Directly counting subarrays with **exactly K distinct elements** is difficult.

Instead, transform the problem:

Count:

- Subarrays with at most K distinct integers
- Subarrays with at most K-1 distinct integers

Then:

Exactly K = At Most K – At Most (K–1)

Why This Transformation Works

Subarrays counted in:

At Most K

- 1 distinct
- 2 distinct
- ...
- K distinct

Subarrays counted in:

At Most K–1

- 1 distinct
- 2 distinct
- ...
- K–1 distinct

Subtracting removes all smaller distinct counts, leaving only:

Exactly K distinct integers

Solution: Sliding Window + At Most K

Solution Explanation

Idea

Create a helper function that counts subarrays having **at most K distinct integers**.

Maintain:

- A sliding window
- Frequency map of elements inside the window

Expand the window by moving the right pointer.

Whenever distinct elements become greater than K:

- Shrink the window from the left
- Remove elements whose frequency becomes zero

Once the window becomes valid:

Every subarray ending at the current position and starting anywhere inside the valid window is also valid.

Add all such subarrays to the answer.

Why Counting ($\text{right} - \text{left} + 1$) Works

Suppose the current window is valid.

If the subarray:

- Left $\rightarrow$ Right

has at most K distinct elements,

then every smaller subarray ending at right and starting between left and right will also have at most K distinct elements.

So all these subarrays are valid and can be counted together.

This avoids checking each subarray individually.

Why This Approach Works

The sliding window always maintains the largest valid window with at most K distinct elements.

For every position:

- All valid subarrays ending there are counted.
- No invalid subarray is included.
- No valid subarray is missed.

Thus the helper correctly counts subarrays with **at most K distinct integers**.

Applying:

At Most K – At Most (K-1)

gives the count of subarrays with **exactly K distinct integers**.

⚡ Complexity Analysis

Helper Function

Time Complexity

- Each element enters the window once.
- Each element leaves the window once.
- HashMap operations are $O(1)$ on average.

$O(N)$

Space Complexity

- Frequency map stores at most K distinct elements.

$O(K)$

Given two strings s and t of lengths m and n respectively, return *the minimum window substring of s such that every character in t (including duplicates) is included in the window*. If there is no such substring, return *the empty string ""*.

The testcases will be generated such that the answer is unique.

💡 Core Intuition

We need the **minimum length valid window**.

A window is valid only when:

- Every character required by t is present.
- Required frequencies are also satisfied.

Key Observation

Instead of checking every substring:

1. Expand the window until it becomes valid.
2. Once valid, try shrinking it from the left.
3. Keep shrinking until the window becomes invalid.
4. Repeat.

This guarantees that every valid window is considered and the minimum one is found.

Solution 1: Brute Force

Generate every possible substring.

For each starting index:

- Extend the substring character by character.
- Maintain frequencies of characters seen.
- Check whether all characters of t are present with required frequencies.

The moment a valid substring is found:

- Update answer if smaller.
- Stop extending further from that starting position since any longer substring cannot be better.

Complexity Analysis

Time Complexity

- Outer loop chooses starting index.
- Inner loop generates substrings.
- Validation checks target frequencies.

$O(N^2 \times M)$

where:

- N = length of source string
- M = number of distinct characters in target

Space Complexity

Frequency maps store characters.

$O(M)$

Solution 2: Sliding Window + HashMap

Core Intuition

Repeatedly validating an entire substring is expensive.

Instead:

- Keep track of character frequencies inside the current window.
- Maintain how many required characters are still missing.

When all required characters are found:

- The window becomes valid.
- Start shrinking from the left to minimize its size.

Solution Explanation

Idea

Maintain:

- Frequency map of target characters.
- Frequency map of current window.
- Counter representing how many target characters are still required.

Expand window:

- Include new characters.
- Reduce missing count whenever a needed character is satisfied.

Shrink window:

- While all requirements are satisfied.
- Update minimum answer.
- Remove unnecessary characters from the left.

Why it Works

The window only becomes valid when every required character frequency is satisfied.

Once valid:

- Any larger window is not better.
- Shrinking helps find the smallest valid window ending at the current position.

Thus every candidate minimum window is examined.

Complexity Analysis

Time Complexity

Each character:

- Enters window once.
- Leaves window once.

$O(N)$

Space Complexity

Two frequency maps are maintained.

$O(M)$

where M is the number of distinct characters stored.

Solution 3: Optimal Sliding Window + Frequency Array

Core Intuition

The HashMap solution is already optimal in terms of traversal.

The only optimization is:

Replace HashMap lookups with a frequency array.

Since characters are ASCII:

- Direct indexing is possible.
 - Operations become faster than HashMap access.
-

Solution Explanation

Idea

Store required frequencies of target characters in an array.

Meaning of frequency array:

- Positive value $\rightarrow$ more occurrences still needed.
- Zero or negative value $\rightarrow$ requirement already satisfied.

Expand window:

- Consume characters.

- Increase matched count whenever a required character is found.

Shrink window:

- Once all required characters are matched.
 - Update minimum window.
 - Remove characters from left.
 - Stop shrinking when a required character becomes missing again.
-

Why it Works

The frequency array always represents the remaining requirement of each character.

When all required characters are matched:

- The current window contains every character needed by t.

Shrinking removes unnecessary characters while preserving validity.

Therefore the minimum valid window is eventually discovered.

Complexity Analysis

Time Complexity

Each character:

- Added once.
- Removed once.

$O(N)$

Space Complexity

Fixed-size frequency array.

$O(1)$